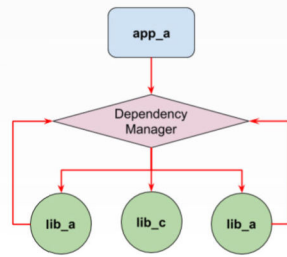


What is Dependency Management?

Software projects rarely work in isolation. In most cases, a project relies on reusable functionality in the form of libraries or is broken up into individual components to compose a modularized system. Dependency management is a technique for declaring, resolving and using dependencies required by the project in an automated fashion.



what is common dependency problems?

Here are some of the most common ones:

- 1. Missing dependencies:** This occurs when a required dependency is not installed or available on the system. This can happen when a developer forgets to include a necessary dependency in the project, or when a package manager fails to install the dependency due to an error or conflict.
- 2. Version conflicts:** This occurs when different dependencies require different versions of the same software library. This can lead to compatibility issues and unexpected behavior in the software. It can also make it difficult to manage dependencies and ensure that the software works correctly.
- 3. Circular dependencies:** This occurs when two or more dependencies rely on each other in a circular manner. This can lead to a situation where it's impossible to install or update the dependencies, and can cause unexpected behavior in the software.
- 4. Security vulnerabilities:** This occurs when a dependency has a known security vulnerability that can be exploited by attackers. It's important to regularly check for security vulnerabilities in dependencies and update them to the latest secure version.
- 5. Deprecated dependencies:** This occurs when a dependency is no longer maintained or updated by the developer, making it obsolete or outdated. This can lead to compatibility issues and security vulnerabilities.

Devops approach for Testing

Devops approach for Testing

In a DevOps approach, testing is an integral part of the software development process, and is typically carried out continuously throughout the development cycle. Here are some key elements of a DevOps approach to testing:

- 1. Automation:** DevOps teams rely heavily on test automation to speed up testing and reduce the risk of errors. Automated tests can be executed quickly and repeatedly, allowing developers to catch bugs early in the development cycle.
- 2. Integration:** Testing is integrated into the development process, with developers and testers working together to identify and resolve issues as soon as they arise. This helps to ensure that the software is always in a releasable state, with any defects caught and fixed quickly.
- 3. Continuous testing:** Testing is carried out continuously throughout the development cycle, with feedback loops in place to ensure that defects are detected and fixed as soon as possible. This helps to reduce the risk of defects being introduced into the software, and ensures that the software is always ready for release.

Traditional testing Vs unit testing

Traditional testing and unit testing are two different approaches to testing software applications.

Traditional testing is a manual testing process that involves testing the entire software application as a whole. Testers create test cases and execute them manually to verify the functionality of the software. Traditional testing is time-consuming, and it can be difficult to identify the root cause of defects.

Unit testing, on the other hand, is an automated testing process that involves testing individual units or components of the software application. Developers write code to test each unit of the software, and these tests are executed automatically. Unit testing is more efficient than traditional testing because it allows developers to identify defects early in the development cycle and fix them quickly.

Continued in next slide.

Traditional testing Vs unit testing

Here are some key differences between traditional testing and unit testing:

- 1. Focus:** Traditional testing focuses on the overall functionality of the software application, while unit testing focuses on individual units or components of the software.
- 2. Timing:** Traditional testing is typically performed at the end of the development cycle, while unit testing is performed early in the development cycle.
- 3. Automation:** Traditional testing is mostly manual, while unit testing is mostly automated.
- 4. Maintenance:** Traditional testing requires manual maintenance of test cases, while unit testing requires minimal maintenance because the tests are automated.

In summary, unit testing is a more efficient and effective testing approach than traditional testing.

Tools for Unit testing

- Selenium
- Cucumber
- Specflow
- QTP
- Appium
- IBM Rational Tester

Selenium – Automation testing tool

Selenium is an open-source automation tool widely used for automating web browsers. It provides a way to automate interactions with web browsers, allowing developers and testers to create automated tests and perform tasks that would otherwise have to be done manually.

Selenium supports multiple programming languages, including Java, Python, Ruby, and C#, and can be used with popular browsers such as Chrome, Firefox, and Safari.

Selenium can be used for a variety of tasks, such as testing web applications, automating repetitive tasks, and web scraping. It has become an essential tool for software development and testing, especially in the field of web development.

<https://www.selenium.dev/>

Components of Selenium

Selenium is mainly built on 3 core components

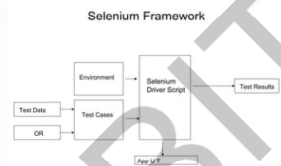
- Selenium IDE (Integrated Development Environment)
 - It is used for record and play back the scripts
- Selenium RC (Remote Control)
 - It is a Server and launches the browser
 - It acts as a API and library of Selenium
- Selenium Grid
 - It is used for launching the multiple browsers with supporting operating system in parallel
 - We can run the scripts on different browsers in parallel.

Testing with Selenium

- We can perform regressing testing with Selenium by writing reusable scripts in the form of functions or methods.
- We can perform retesting with looping concepts
- We can validate the application by using the conditional statements
- We can perform database testing

Selenium Framework

- The execution of the script starts, based on the Selenium Driver script
- Selenium Driver script takes the URL from the environment folder and test cases from the test cases folder as an input
- In the test cases folder all the test cases will be stored and these test cases will take the test data from the 'test data' folder
- After completion of the execution the results are stored in the results folders. It contains pass or fail.



Need for Code Inspection & Analysis

Code Quality

Code quality is an extremely important parameter for software quality deliverables and affects the overall success of a software organization. It is critical to ensure utmost quality of delivered software as code quality defines how safe, secure and reliable the software is.

Code quality depends on a number of different attributes and requirements, primarily determined and prioritized by the business needs. Essentially, a good quality code

- Does what it should.
- Follows a consistent style.
- Is easy to understand.
- Has been well-documented.
- Can be tested.

Key Measure for high quality code

- **Reliability**
Reliability measures the system's probability to run without failure over a specific period of operation. It relates to the number of defects and availability of the software.
- **Maintainability**
Maintainability measures how easy is the system maintenance. It relates to the size, consistency, structure, and complexity of the codebase. Ensuring that source code is maintainable depends on a number of factors, such as testability and understandability.
- **Testability**
Testability measures how well the software can be tested. It relies on the extent to which you can control, observe, isolate, and automate testing.
- **Portability**
Portability measures how usable or portable the same software is across different environments. Platform independence is the key factor.
- **Reusability**
Reusability measures whether existing code can be used again or not. Modular and loosely coupled code, is easy to reuse.

How to Improve Code Quality

Let us discuss some ways you can improve code quality:

- **Coding Standards**
using a static code analyzer helps.
- **Code Analysis before Code Reviews**
Quality should be a priority from the very start of development and that's why it's important to analyze code before code reviews begin. In fact, it's best to analyze code as soon as it's written. The earlier the defects are detected, the easier and faster it is to resolve them.
- **Code Review Best Practices**
Good code reviews help improve overall software quality. We will discuss Code reviews in detail shortly.
- **Refactor Legacy Code**
One way to improve the quality of an existing codebase is through refactoring.

Static Code Analysis

Static code analysis is a set of algorithms and techniques used to analyze source code in order to automatically find potential errors or poor coding practices.

Static code analysis, also commonly known as "white-box" testing, tests applications in non-runtime environments. It is a proven technique which covers entire codebase and identifies all the vulnerabilities.

The tasks performed in the static code analysis can be divided into 3 categories:

- Detecting errors in programs
- Recommendations on code formatting
- Software Metrics computation

SonarQube

SonarQube is a web-based open source platform by SonarSource, used to measure and analyse the source code quality. Code quality analysis makes your code more reliable and more readable.

SonarQube can inspect and evaluate anything that affects code base, from minor styling details to critical design errors. Hence, it is a great aid to software application developers in identifying the issues and their impact.

Key Concepts in Static Code Analysis

Code Smell: It is basically an indicator of something fishy in the code – “a hint that something has gone wrong somewhere in your code

Code coverage: It is a measure which describes the degree of which the source code of the program has been tested. It is one form of white box testing which finds the areas of the program not exercised by a set of test cases.

Test coverage: It is a Software Testing metric used to measure the amount of testing performed by a particular set of tests.

Vulnerabilities: It is a cyber-security term that refers to a flaw or loophole in the system that makes it vulnerable to attack.

How SonarQube helps in testing?

Developers

Regularly usage of SonarQube helps developers identify the coding standard violations at the time of coding and they learn to adhere to coding standards from the beginning.

Technical management

SonarQube supports easy integration with version control system to track down the details regarding each code update. This also helps to identify a particular developer's performance and his coding practices.

Non-technical management

Non-Technical management will be keen to know and measure code quality. They may not understand the complexity and duplications etc but with the matrix and total statistics about the project health, it becomes easy for them to make key decisions about projects.

IMP Note to Self





BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Thank you

BITS Pilani, Pilani Campus





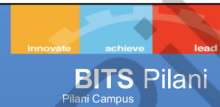
BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Introduction to DevOps

Slides edited /compiled by K V Vamsi Krishna
for DevOps faculty team.

IMP Note to Self



CSIW ZG515, Introduction to DevOps CS 07 : DevOps – Continuous Integration and Continuous Delivery

IMP Note to Students

- ☐ It is important to know that just login to the session does not guarantee the attendance.
- ☐ Once you join the session, continue till the end to consider you as present in the class.
- ☐ IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- ☐ **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**



Coverage

Continuous Integration and Continuous Delivery

- Overview of Continuous Integration-Version control, automated build, Test
- Prerequisites for Continuous Integration
- Continuous Integration Practices
- Team responsibilities
- Using Continuous Integration Software (Jenkins as an example tool)
- Jenkins Architecture

BITS Pilani, Pilani Campus

Continuous Integration

Continuous Integration

Continuous Integration Explained

- Continuous integration is a DevOps software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run.
- Continuous integration most often refers to the build or integration stage of the software release process and entails both an automation component (e.g. a CI or build service) and a cultural component (e.g. learning to integrate frequently).
- The key goals of continuous integration are to find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

Why is Continuous Integration Needed?

- Merge their changes to the master branch once their work was completed.
- This made merging code changes difficult. In the past, developers on a team might work in isolation for an extended period of time and only and time-consuming, and also resulted in bugs accumulating for a long time without correction.
- These factors made it harder to deliver updates to customers quickly.

Continuous Integration

How does Continuous Integration Work?

- Developers frequently commit to a shared repository using a version control system such as Git.
- Prior to each commit, developers may choose to run local unit tests on their code as an extra verification layer before integrating.
- A continuous integration service automatically builds and runs unit tests on the new code changes to immediately surface any errors.



Continuous Integration



Continuous Integration, Continuous Deployment and Continuous Delivery

Continuous Integration

- It is the practice of integrating changes from different developers in the team into a mainline as early as possible, in best cases several times a day.
- This makes sure the code individual developers work on doesn't divert too much.
- When you combine the process with automated testing, continuous integration can enable your code to be dependable.

Continuous Deployment

- It is closely related to continuous integration and refers to keeping your application deployable at any point.
- It involves frequent, automated deployment of the master branch to a production environment following automated testing.

Continuous Delivery

- It is the discipline of being able to reliably release application changes (code) at any time.
- Beyond making sure your application passes automated tests it has to have all the configuration necessary to push it into production.
- CD encompasses everything from ideation to build to readiness to delivery to production.
- continuous delivery includes continuous integration practices.

Automation is implicit. There is no CI+ CD or CI/CD.

Prerequisites for Continuous Integration

Prerequisites for Continuous Integration

The basic prerequisites for implementing continuous integration include:

- Automating builds;
- Automating testing;
- More frequent commits to a single source code repository, and
- Providing visibility of the process and real-time access to CI status to the team.
- Teams that don't practice CI yet should take small steps, continuously improve, and iterate on code and process in a way that helps the organization grow.
- On every step in the journey to full CI/CD, the development team's productivity will rise, as well as the velocity of the entire business.

Benefits of Continuous Integration



CI provides numerous benefits for your software development team including improving developer productivity, automating the development process, improving code quality and software reliability.

Improved developer productivity

- Continuous integration frees developers from manual tasks and the pains of integrating their code with other parts of the system.
- Instead, they can focus on programming the logic that delivers the features the business needs. Or they can use the time CI has saved them to invest in professional growth.
- Teams that work in a fast CI feedback loop can deliver more value to customers than teams that struggle to integrate each other's work or suffer from a slow pipeline that doesn't scale.

Deliver working software more often

- Continuous integration is a way for your team to automatically build and test every single change in the source code.
- This is the foundation that enables your broader software delivery process to be efficient, resilient, fast, and secure.

Benefits of Continuous Integration



Find bugs earlier, fix them faster

- The automated testing process can include many different types of checks:
 - Verify code correctness;
 - Validate application behavior from a customer's perspective;
 - Compare coding style with industry-standard conventions;
 - Test code for common security holes;
 - Detect security updates in third-party dependencies;
- Measure test coverage: how much of your application's features are covered by automated tests.
- Building these tests into your CI pipeline, measuring and improving the score in each is a guaranteed way to maintain a high quality of your software.
- A CI tool provides instant feedback to developers on whether the new code they wrote works, or introduces bugs or regression in quality.
- Mistakes caught early on are the easiest to fix.

Continuous Integration Practices



Continuous Integration Practices



- Maintain a code repository
- Automate the build
- Make the build self-testing
- Everyone commits to the baseline every day
- Every commit (to baseline) should be built
- Keep the build fast
- Test in a clone of the production environment
- Make it easy to get the latest deliverables
- Everyone can see the results of the latest build
- Automate deployment



Continuous Integration Practices



Maintain a code repository

- This practice advocates the use of a revision control system for the project's source code.
- All artifacts required to build the project should be placed in the repository.
- In this practice and the revision control community, the convention is that the system should be buildable from a fresh checkout and not require additional dependencies. It is preferred for changes to be integrated rather than for multiple versions of the software to be maintained simultaneously.
- The mainline should be the place for the working version of the software.

Automate the build

- A single command should have the capability of building the system.
- Many build tools have existed for many years. Other more recent tools are frequently used in continuous integration environments.
- Automation of the build should include automating the integration, which often includes deployment into a production-like environment.
- In many cases, the build script not only compiles binaries but also generates documentation, website pages, statistics and distribution media

Make the build self-testing

- Once the code is built, all tests should run to confirm that it behaves as the developers expect it to behave.
- Everyone commits to the baseline every day
- By committing regularly, every committer can reduce the number of conflicting changes.
- Checking in a week's worth of work runs the risk of conflicting with other features and can be very difficult to resolve.
- Early, small conflicts in an area of the system cause team members to communicate about the change they are making.
- Committing all changes at least once a day is generally considered part of the definition of Continuous Integration.
- In addition, performing a nightly build is generally recommended. These are lower bounds; the typical frequency is expected to be much higher.

Continuous Integration Practices



Every commit (to baseline) should be built

- The system should build commits to the current working version to verify that they integrate correctly.
- A common practice is to use Automated Continuous Integration, although this may be done manually.
- Automated Continuous Integration employs a continuous integration server or daemon to monitor the revision control system for changes, then automatically run the build process.

Every bug-fix commit should come with a test case

- When fixing a bug, it is a good practice to push a test case that reproduces the bug. This avoids the fix to be reverted, and the bug to reappear, which is known as a regression.

Keep the build fast

- The build needs to complete rapidly so that if there is a problem with integration, it is quickly identified.

Test in a clone of the production environment

- Having a test environment can lead to failures in tested systems when they deploy in the production environment because the production environment may differ from the test environment in a significant way.
- However, building a replica of a production environment is cost-prohibitive.
- Instead, the test environment or a separate pre-production environment ("staging") should be built to be a scalable version of the production environment to alleviate costs while maintaining technology stack composition and nuances.

Continuous Integration Practices



Make it easy to get the latest deliverables

- Making builds readily available to stakeholders and testers can reduce the amount of rework necessary when rebuilding a feature that doesn't meet requirements.
- Additionally, early testing reduces the chances that defects survive until deployment. Finding errors earlier can reduce the amount of work necessary to resolve them.
- All programmers should start the day by updating the project from the repository. That way, they will all stay up to date.

Everyone can see the results of the latest build

- It should be easy to find out whether the build breaks and, if so, who made the relevant change and what that change was.

Automate deployment

- Most CI systems allow the running of scripts after a build finishes.
- In most situations, it is possible to write a script to deploy the application to a live test server that everyone can look at.
- A further advance in this way of thinking is continuous deployment, which calls for the software to be deployed directly into production, often with additional automation to prevent defects or regressions.

Team Responsibilities



Team Responsibilities

- Develop CI/CD principles
- Review and modify CI/CD principles, iteratively
- Maintain CI/CD tools/platforms (if applicable)
- Develop and maintain pipeline configurations
- Automate processes

Team Responsibilities

Develop CI/CD principles

- Adoption of CI/CD principles produces huge gains in efficiencies.
- These practices are composed of processes that are defined by stakeholders and facilitate optimized software development. The Team will be an authority on everything "CI/CD".
- As an authority, they will be actively involved in the development and implementation of these related processes.

Review and modify CI/CD principles

- Once practices are established, it's critical that teams continually revisit them to verify that they support current operations optimally.
- Team will be in a unique position to develop a deep understanding of how respective teams are operating and interacting with the other units.
- They'll have insight into what's working and what's not with the added ability to determine and offer corrective actions.
- Iterative reviews will provide consistent visibility into the processes that drive software development and surface options for improvement.

Manage CI/CD tools/platforms

- The next step after establishing and adopting CI/CD practices and principles is leveraging CI/CD tooling and platforms that will facilitate the execution of related processes.
- CI/CD platforms bring our processes to life and provide the automation that executes specified pipelines. Implementing and managing CI/CD tooling requires a combination of skills, DevOps for sure, but in this aspect a bit more emphasis on Ops is appropriate.
- Ensuring that CI/CD tooling operates consistently definitely falls under the "operations" wheelhouse. This particular duty is also dependent on architecture and operational landscapes.

Team Responsibilities

Develop and maintain pipeline configurations

- All CI/CD tooling requires a form of pipeline configuration which is the mechanism that specifies the various steps and segments to execute within your CI/CD processes.
- Configuring pipelines is completed in different ways depending on the CI/CD tool being used. CI/CD pipelines must be specified and maintained, and this duty will fall squarely on the Team.
- Developing, managing, and executing CI/CD pipelines is the biggest responsibility that this role owns. The pipeline configurations orchestrate the execution of the steps specified. In other words, pipeline configurations are how we program the CI/CD tools to do what we want.
- Team will ensure that pipelines are properly defined and performing optimally. They will have a deep understanding of the pipeline's goals and the transactions occurring within them.
- They will interface with DevOps teams to coordinate and optimize the steps that execute within the pipeline.
- The CI/CD engineer will document the pipelines so that all of the interested parties are aware of how their software is built, tested, secured, and deployed.

Automate processes

- Automation is at the core of CI/CD. It facilitates processing pipeline commands, inter-platform connectivity, and valuable integrations with 3rd party services.
- If you peel back the layers, at the core, Team will design and manage the automation that powers continuous integration and delivery operations.
- The level of experience can vary, but a foundational knowledge of automation will be required to be an effective Team.

Using Continuous Integration Software

Using Continuous Integration Software

Jenkins is an open-source implementation of a Continuous Integration server written in Java.

- It streamlines multiple actions for Continuous Integration (CI) in a software project. It is a build server and is a tool that provides this functionality.
- It works with multiple programming languages and can run on various platforms (Windows, Linux, and macOS).
- It is widely used as a CI (Continuous Integration) & CD (Continuous Delivery) tool.
- It has vast community support, and many plugins are available for integrating it with other tools like Slack, GitHub, and Docker. Also, anyone can develop a Jenkins plugin and contribute to it.
- By using Jenkins, software companies can accelerate their software development process, as it can automate a build and run tests to ensure the functionality is working fine.
- Jenkins supports the entire SDLC, including building, testing, documenting the software, and deploying.

Benefits of using Jenkins CI

Reduced Development Cycle

- Since every commit is built and tested, it allows releasing new features to the user faster and with fewer errors.

Shorter Time to Integrate Code

- Before the use of Jenkins CI, integration of code was done manually, thus taking a few days.
- In some cases, it might happen that the code is not running, and it is hard to debug as it might have gone through various commits in the repository.
- Integrating code after every commit ensures that the functionality is not broken after a commit.

Faster Feedback Loops

- Developers get feedback and improve the code whenever a test breaks during a commit.
- Otherwise, debugging the issue can be very difficult, given teams would not be sure which commit resulted in the bug.

Automated Workflow

- Teams should not worry about running a manual test for each commit.
- The Jenkins CI pipeline checks the latest code and builds the code along with the tests.
- The test can deploy the project in a specific environment if it is green.
- It can notify the developer by breaking the build.

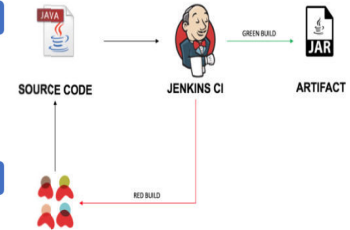
How to create a CI Pipeline in Jenkins

1. Declarative Pipeline

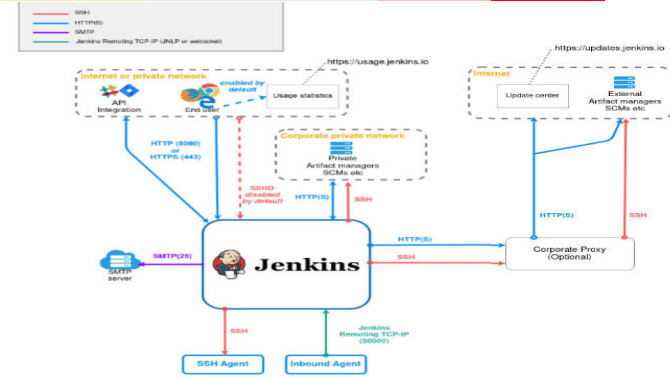
- The Jenkins pipeline can be written declaratively with the "Jenkins syntax".
- Jenkins syntax is a **Groovy DSL** (Domain Specific Language) used to write the pipeline stages.
- The DSL internally gets converted into XML to make the Jenkins pipeline.
- A "stage" in a pipeline is a group of steps to be executed.

2. Jenkins Job Builder

- JJB (Jenkins job builder) is a pipeline creation tool using YAML configuration.
- The Jenkins pipeline can be written in YAML, and with the help of the JJB tool, it converts the YAML configuration into XML format and pushes it into Jenkins to create pipelines.
- It is comparatively easier to write YAML configuration than full-fledged code in Jenkins syntax.
- It also enables the use of "Pipeline as Code" so that a pipeline code can be pushed into a git repo, and the pipeline gets created or updated in Jenkins.
- The tool also helps with the templating of the pipeline code.



Jenkins Architecture





IMP Note to Self



Thank you



BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Introduction to DevOps

Slides edited /compiled by Akanksha Bharadwaj
for DevOps faculty team.

CSIW ZG515, Introduction to DevOps CS 08 : DevOps – Artifacts Management

IMP Note to Self



IMP Note to Students

- ☐ It is important to know that just login to the session does not guarantee the attendance.
- ☐ Once you join the session, continue till the end to consider you as present in the class.
- ☐ IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- ☐ **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**

BITS Pilani, Pilani Campus



Coverage

Artifacts Management

- Overview of Artifacts management
- Setting up the Continuous Integration pipeline
- Continuous delivery to staging environment or the pre-production environment
- Self-healing systems

BITS Pilani, Pilani Campus

Artifact Management

What are Artifacts?

Artifacts in the context of software development refer to any tangible output generated during the software development process. These can include:

Compiled Code: The result of compiling source code into machine-readable instructions or bytecode.

Executable Files: Programs that can be run directly by a computer's operating system.

Libraries: Collections of precompiled routines, functions, or classes that can be used to add specific functionality to software applications.

Documentation: Manuals, guides, or reference materials that provide information about how to use, configure, or maintain the software.

Configuration Files: Files containing settings or parameters that control the behavior of the software or its components.

Assets: Any additional files or resources required by the software, such as images, audio files, or data files.

BITS Pilani, Pilani Campus

Artifact Management

- Artifact management refers to the systematic management of various outputs generated during the software development lifecycle.
- These outputs, known as artifacts, encompass a wide range of items including compiled code, executables, libraries, documentation, configuration files, and more.
- The primary goal of artifact management is to ensure that these artifacts are organized, versioned, stored, and distributed efficiently to support the software development process.

BITS Pilani, Pilani Campus

Objectives

An Artifact repository should be

- Secure
- Scalable
- Versioned
- Searchable
- Stable
- Highly Available

BITS Pilani, Pilani Campus

Importance of Artifact Management

- Reproducibility
- Traceability
- Collaboration
- Version Control
- Dependency Management
- Automation

BITS Pilani, Pilani Campus

Benefits of Artifact Management

- The Binary Repository Manager: Artifact repositories allow teams to easily find the correct version of a given artifact
- Which means developers can push to and pull from the artifact repository as part of the DevOps workflow
- Single source of Truth: Artifact repositories improve the way teams work together by ensuring everyone using the correct files
- It helps CI/CD to be more reliable

BITS Pilani, Pilani Campus

Artifacts Management in Software Development

- Documentation => Version Control
- The source code => Source Code Version Control
- The compiled application => Dependency Management
- A deployable package => Provisioning and Configuration Management

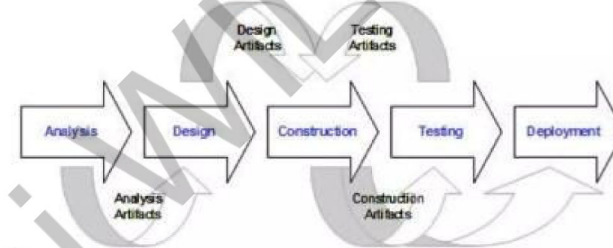
BITS Pilani, Pilani Campus

Main Features

- Versioning Support
- Reproducibility
- Access Control
- Promotion
- Change Management

BITS Pilani, Pilani Campus

Artifacts in Software Development Life Cycle



BITS Pilani, Pilani Campus

CI- What does it really mean?

- At a regular frequency (ideally at every commit), the system is:
 - Integrated
 - All changes up until that point are combined into the project
 - Built
 - The code is compiled into an executable or package
 - Tested
 - Automated test suites are run
 - Archived
 - Versioned and stored so it can be distributed as is, if desired
 - Deployed
 - Loaded onto a system where the developers can interact with it

BITS Pilani, Pilani Campus

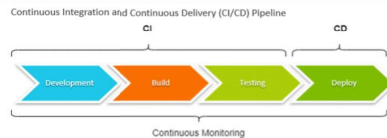
What is a Pipeline?

- A deployment pipeline consists of the steps that are taken between a developer, committing code and the code actually being promoted into normal production, while ensuring high quality.
- CI/CD, in layman's terms, is a step-by-step approach to continuously and consistently code, build, test, release, and monitor a software product through automation.
- Pipelines are pre-defined jobs that help understand what needs to be accomplished and when it has to be done. Jobs are executed in parallel as much as possible to speed up and accelerate the delivery.

BITS Pilani, Pilani Campus

Setting up Pipeline

- A typical CI/CD pipeline would consist of stages where code gets pushed to the repository; the build gets triggered, build is tested and deployed to the production environment.
 - Building CI/CD pipelines are completely tailor-made based on the needs and requirements and could have multiple stages and jobs, and could be complex and comprehensive.
- Typically building a CI/CD pipeline consists of the following phases/stages.
- **Code:** Checked into the repository
 - **Build:** Build is triggered and deployed in a test environment
 - **Test:** Automated tests are executed
 - **Deploy:** Code is deployed to stage, and production environments.



BITS Pilani, Pilani Campus

Characteristics of a Pipeline

To build effective pipelines, it is important to ensure that they have the below characteristics:

- Reliability
- Efficiency
- Scalability
- Flexibility
- Transparency
- Maintainability
- Security

BITS Pilani, Pilani Campus

What is Continuous Delivery Pipeline?

- Continuous delivery (CD) is a software development method that makes it possible to automatically build, test, and deploy new versions of an application. Continuous delivery is based on continuous integration (CI) practices (together they are called CI/CD), but adds the ability to fully automate software deployment to staging and production environments.
- A continuous delivery pipeline is a structured, automated process that typically starts with a developer who commits new code to a repository. This code triggers a CI build process, which might be integrated with container registries or binary repositories. The new build is subjected to automated tests, might be deployed to a staging environment for additional testing, and can then be deployed to production with the push of a button.

BITS Pilani, Pilani Campus

Stages of Continuous delivery Pipeline

- Build/Develop
- Commit
- Test
- Stage
- Deploy

BITS Pilani, Pilani Campus



1. Build/Develop: A build/develop process performs the following:

- Pulls source code from a public or private repository.
- Establishes links to relevant modules, dependencies, and libraries.
- Builds (compiles) all components into a binary artifact.

BITS Pilani, Pilani Campus



2. Commit: It tasks typically run as a set of jobs, including:

- Compile the source code
- Run the relevant commit tests
- Create binaries for later phases
- Perform code analysis to verify health
- Prepare artifacts like test databases for later phases

These jobs run on a build grid, a facility provided by continuous integration (CI) servers. It helps ensure that the commit stage completes quickly, in less than five minutes, ideally, and no longer than ten minutes.



3. Test: During the test phase, the completed build undergoes comprehensive dynamic testing. It occurs after the source code has undergone static testing. Dynamic tests commonly include Unit/Functional testing & Regression testing

4. Stage: The staging phase involves extensive testing for all code changes to verify they work as intended, using a staging environment, a replica of the production (live) environment. It is the last phase before deploying changes to the live environment.

The staging environment mimics the real production setting, including hardware, software, configuration, architecture, and scale. You can deploy a staging environment as part of the release cycle and remove it after deployment in production.



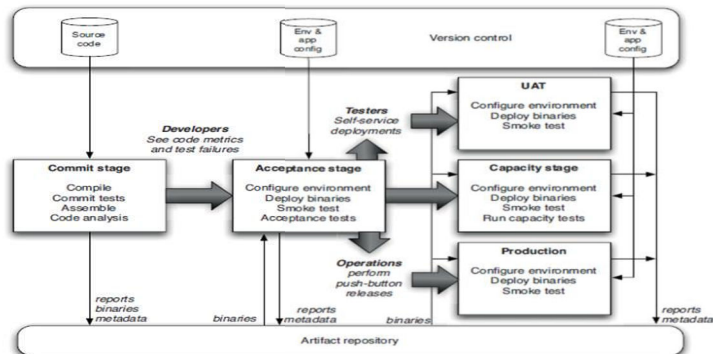
5. Deploy: The deployment phase occurs after the build passes all testing and becomes a candidate for deployment in production. A continuous delivery pipeline sends the candidate to human teams for approval and deployment. A continuous deployment pipeline deploys the build automatically after it passes testing.

Deployment involves creating a deployment environment and moving the build to a deployment target. Typically, developers automate these steps with scripts or workflows in automation tools. It also requires connecting to error reporting and ticketing tools. These tools help identify unexpected errors post-deployment and alert developers, and allow users to submit bug tickets.



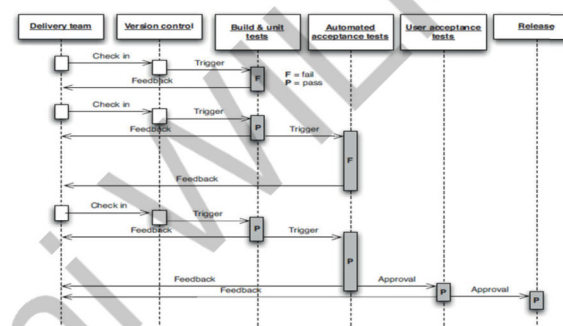


Basic Deployment Pipeline



BITS Pilani, Pilani Campus

Deployment Pipeline



BITS Pilani, Pilani Campus

Self Healing System

A self-healing system can discover errors in its functioning and make changes to itself without human intervention, thereby restoring itself to a better-functioning state. There are three levels of self-healing systems, each of which has its own size and resource requirements

Application Level

In typical applications, problems are documented in an 'exceptions log' for further examination. Most problems are minor and can be ignored. Serious problems may require the application to stop (for example, an inability to connect to a database that has been taken offline).

By contrast, self-healing applications incorporate design elements that resolve problems. For example, applications that use [Akka](#) arrange elements in a hierarchy and assign an actor's problems to its supervisor. Many such libraries and frameworks facilitate applications that self-heal by design.

BITS Pilani, Pilani Campus





System Level

Unlike application level self-healing, system level self-healing does not depend on a programming language or specific components. Rather, it can be generalized and applied to all services and applications, independent of their internal components.

The most common system level errors include process failures (often resolved by redeploying or restarting) and response time issues (often resolved by scaling and descaling). Self-healing systems conduct health checks on different components and automatically attempt fixes (such as redeploying) to recuperate to their desired states.

Hardware Level

Hardware level self-healing redeploys services from an unhealthy node to a healthy one. It also conducts health checks on different components. Since true hardware level self-healing (for example, a machine that can heal failed memory or repair a broken hard disk) does not exist, current hardware level solutions are essentially system level solutions.



Designing Self-Healing Systems

Get to know your application. This is the most important principle. It's easy to dive in and start automating self-healing mechanisms, but you'll waste time if you don't understand the production problems your application is having before you begin. First, setup automated alerts to see which error scenarios are most common.

Prevention is better than cure. Sure, it's nice to be able to recover from error scenarios automatically, but it's better to prevent the error scenarios wherever possible. Take a holistic look at problems you detect so that you can identify and fix the root causes where possible.

A primary goal of self-healing systems should be happier development teams. Self-healing systems bring benefits to users and operations. But an often-overlooked benefit is that an application that fixes itself reduces the support burden on the development team. Less rote and menial work means happier developers.

Five Point Roadmap

- Use immutable infrastructure as code
- Automate testing to keep the codebase efficient
- Deploy holistic monitoring systems
- Employ leading-edge smart alerts, triggers and prescriptive analytics
- Think deeply about how the system can improve self-learning

BITS Pilani, Pilani Campus

Key characteristics and components of self-healing systems



Monitoring: Self-healing systems continuously monitor their environment, collecting data on various metrics, performance indicators, and states of the system components. Monitoring may include metrics such as CPU usage, memory consumption, network traffic, and application response times.

Anomaly Detection: Self-healing systems use advanced algorithms and analytics to analyze the collected data and detect anomalies or deviations from normal behavior. Anomalies may indicate potential failures, performance degradation, security threats, or other issues that require attention.

Diagnosis: Once an anomaly is detected, self-healing systems analyze the root cause of the issue to determine its underlying cause or source. This may involve correlating data from multiple sources, performing diagnostic tests, or using machine learning algorithms to identify patterns and trends.

Decision Making: Based on the diagnosis, self-healing systems make informed decisions on how to respond to the detected anomaly. This may involve choosing the most appropriate course of action to address the issue, such as restarting a failed service, reallocating resources, or adjusting system configurations.

Automated Remediation: Self-healing systems take automated actions to remediate the detected issues and restore the system to a healthy state. Remediation actions may include restarting failed services, scaling resources up or down, applying patches or updates, or reconfiguring system settings.

Feedback Loop: Self-healing systems incorporate feedback mechanisms to evaluate the effectiveness of their actions and learn from past experiences. Feedback may come from monitoring data, user interactions, or external systems, enabling continuous improvement and optimization of the self-healing capabilities.

Resilience: Self-healing systems are designed to be resilient in the face of failures, disruptions, or attacks. They employ redundancy, fault tolerance, and failover mechanisms to ensure that critical functions can continue even in the presence of faults or failures.

Adaptability: Self-healing systems are adaptable and can adjust their behavior or configurations dynamically in response to changing conditions, requirements, or threats. They may use adaptive algorithms or policies to optimize their performance and effectiveness over time.

BITS Pilani, Pilani Campus



IMP Note to Self



BITS Pilani, Pilani Campus





BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Thank you

BITS Pilani, Pilani Campus





BITS Pilani
Pilani | Dubai | Goa | Hyderabad | Mumbai
**WORK INTEGRATED
LEARNING PROGRAMMES**



Introduction to DevOps

Slides edited /compiled by Dr. Selvaraj Kesavan
for DevOps faculty team.

IMP Note to Self



CSIW ZG515, Introduction to DevOps

CS 09 : DevOps – Continuous Deployment

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
- Once you join the session, continue till the end to consider you as present in the class.
- IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
- **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**



Coverage

- Continuous Deployment
 - Introduction to continuous deployment
 - Human-free deployments
 - Implementing and Automating the deployment process
 - Deploying it to testing environments
 - Releasing software into production
 - Environment-based release patterns
- Reference
 - T1- Chapter 6, 12
 - T2- Chapter 10
 - R2- Chapter 3, 4

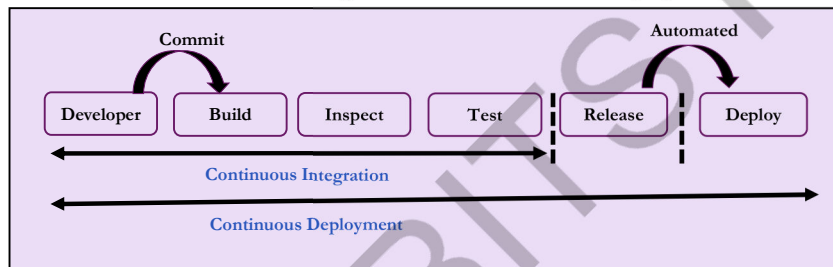
BITS Pilani, Pilani Campus

Continuous Deployment

- Deployment is the process of placing a version of a service into production.
- The initial deployment of a service can be viewed as going from no version of the service to the initial version of the service.
- initial deployment happens only once for most systems and new versions happen frequently.
- Goal of a deployment is to place an upgraded version of the service into production with minimal impact to the users of the system, be it through failures or downtime.
- Redeployment requires to fix an error, to improve some quality of the service, or to add a new feature
- **Release:** A business term that defines functionality being available to an end-user
- **Deployed:** The functionality is successfully placed and running in Production environment

Continuous Deployment

Continuous Deployment is a software development practice that aims to automate the release process and enable frequent and reliable software deployments



BITS Pilani, Pilani Campus

Deployment - Considerations

Deployment
Consideration

- Compatibility
 - Backward
 - Forward
- Canary Testing
- Rollback or Roll forward
- Relevant process & Tools

BITS Pilani, Pilani Campus

Deployment - Challenges

- Build and operations teams waiting for documentation or fixes
- Testers waiting for “good” builds of the software
- Development teams receiving bug reports weeks after the team has moved on to new functionality
- Discovering, towards the end of the development process, that the application’s architecture will not support the system’s nonfunctional requirements

This leads to software that is undeployable because it has taken so long to get it into a production-like environment, and buggy because the feedback cycle between the development team and the testing and operations team is so long

BITS Pilani, Pilani Campus

Deployment –Approach

- End-to-End approach to delivering software
- Deployment of Application should be **easy & one click to go**
- A powerful feedback loop; rapid feedback on both the code and the deployment process
- Lowering the risk of a release and transferring knowledge of the deployment process to the development team
- Testing teams deploy builds into testing environments themselves, at the push of a button
- Operations can deploy builds into staging and production environments at the push of a button
- Developers can see which builds have been through which stages in the release process, and what problems were found
- Automate Build, Deploy, Test and Release System

Note: As a result, everybody in the delivery process gets two things: access to the things they need, when they need them, and visibility into the release process to improve feedback so that bottlenecks can be identified, optimized, and removed.

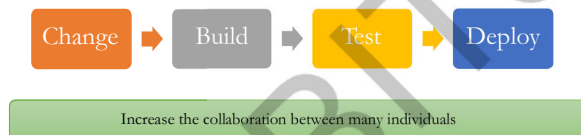
This leads to a delivery process which is not only faster but also safer and risk free

BITS Pilani, Pilani Campus

Deployment Pipeline

What is Deployment Pipeline

- A deployment pipeline is a series of automated steps and stages that code changes go through in order to be deployed to a production environment.
- The purpose of a deployment pipeline is to ensure that code changes are built, tested, and validated before being deployed to production. It helps catch issues early in the development process and ensures that only high-quality and reliable software reaches end-users.



BITS Pilani, Pilani Campus

Human Free Deployments

Automating Deployment and Release

- Reduces issues due to manual mistakes
- Audit logs

Benefits

- With automated deployment and release, the process of delivery becomes available to everyone
- Developers, testers, and operations teams no longer need to rely on ticketing systems and email threads to get builds deployed so they can gather feedback on the production readiness of the system
- Testers can decide which version of the system they want in their test environment without needing to be technical experts themselves, nor relying on the availability of such expertise to make the deployment
- Sales people can access the latest version of the application with the killer feature that will swing the deal with a client
- An important reason for the reduction in risk is the degree to which the process of release itself is rehearsed, tested, and perfected
- Since you use the same process to deploy your system to each of your environments and to release it, the deployment process is tested very frequently - perhaps many times a day

BITS Pilani, Pilani Campus

Implementing and Automating Deployment Process

- Creating Release Strategy
- Release Plan
- Release Products
- Deploying and Promoting

BITS Pilani, Pilani Campus

Creating a Release Strategy

- Application's stakeholders to meet up during the project planning process. The point of their discussions should be working out a common understanding concerning the deployment and maintenance of the application throughout its lifecycle.
- Shared understanding is then captured as the release strategy and will be updated and maintained by the stakeholders throughout the application's life.
- Considerations including
 - Parties in charge of deployments to each environment, as well as in charge of the release
 - An asset and configuration management strategy
 - A description of the technology used for deployment. This should be agreed upon by both the operations and development teams
 - A plan for implementing the deployment pipeline.
 - An enumeration of the environments available for acceptance, capacity, integration, and user acceptance testing, and the process by which builds will be moved through these environments

BITS Pilani, Pilani Campus

Creating a Release Strategy

- A description of the processes to be followed for deployment into testing and production environments, such as change requests to be opened and approvals that need to be granted.
- Requirements for monitoring the application, including any APIs or services the application should use to notify the operations team of its state.
- Description of the integration with any external systems. At what stage and how are they tested as part of a release? How do the operations personnel communicate with the provider in the event of a problem?
- A disaster recovery plan so that the application's state can be recovered following a disaster
- How upgrades to the production environment will be handled, including data migration
- How application support will be managed
- How the initial deployment to production works
- How fixing defects and applying patches to the production environment will be

BITS Pilani, Pilani Campus

Release Plan

- A vital component of the release strategy is the release plan describing how releases are performed
- The first release is usually the one that carries the highest risk; it needs careful planning.
- The results of this planning may be automated scripts, documentation, or other procedures needed reliably and repeatedly deploy the application into the production environment.
- Release plan should include
 - The steps required to deploy the application for the first time
 - How to smoke-test the application and any services it uses as part of the deployment process
 - The steps required to back out the deployment should it go wrong
 - The steps required to back up and restore the application's state
 - The steps required to upgrade the application without destroying the application's state
 - The steps to restart or redeploy the application should it fail
 - The location of the logs and a description of the information they contain
 - The methods of monitoring the application
 - The steps to perform any data migrations that are necessary as part of the release
 - An issue log of problems from previous deployments, and their solutions

BITS Pilani, Pilani Campus

Releasing Products

- To release software product, other issues is software destined to be released as a commercial product.
 - Pricing model
 - Licensing strategy
 - Copyright issues around third-party technologies used
 - Packaging
 - Marketing materials—print, web-based, podcasts, blogs, press releases, conferences, etc.
 - Product documentation
 - Installers
 - Preparing sales and support teams

BITS Pilani, Pilani Campus

Deploying and Promoting Application

- Key to deploying any application in a reliable, consistent manner is constant practice: Use the same process to deploy to every environment, including production.
- Automating the deployment should start with the very first deployment to a testing environment. Instead of manually pulling the pieces of software into shape, write a simple script that does the job.
 - First Deployment
 - Modeling Your Release Process and Promoting Builds
 - Promoting Configuration
 - Orchestration
 - Deployments to Staging Environments

BITS Pilani, Pilani Campus

First Deployment

- First deployment of any application should happen in the first iteration when showcase first stories or requirements to the customer.
- Choose one or two stories or requirements that are of high priority but very simple to deliver in your first iteration .
- Use this showcase as a reason to make the application deployable to a production like showcase environment (UAT)
- one of the principal goals of the first iteration of a project is to get the early stages of our deployment pipeline running and to be able to deploy and demonstrate something, no matter how small, at the end.
- At the end of this initial iteration, the running end to end pipe line should be in place

BITS Pilani, Pilani Campus

First Deployment

It should have

- Deployment pipeline's commit stage
- A production-like environment to deploy to
- An automated process that takes the binaries created by your commit stage and deploys them into the environment
 - A simple smoke test that verifies that the deployment worked and the application is running
- If production environment is a cluster, build a small, limited cluster for staging environment
- If production environment is a distributed one with many different nodes, make sure production-like test environment has at least one separate process to represent each class of process boundary.

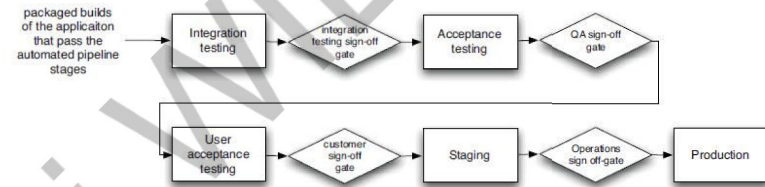
BITS Pilani, Pilani Campus

Modeling Your Release Process and Promoting Builds

- As application grows and becomes more complex, it is important to have robust deployment pipeline implementation.
- Since deployment pipeline should model test and release process, you need first to work out what this process is.
- It is important to capture
 - What stages a build has to go through in order to be released (for example, integration testing, QA acceptance testing, user acceptance testing, staging, production)
 - What the required gates or approval are
 - For each gate, who has the authority to approve a build passing through that gate

BITS Pilani, Pilani Campus

Modeling Your Release Process and Promoting Builds



BITS Pilani, Pilani Campus

Promoting Configuration

- It is not just the binaries that need to be promoted. The configuration of the environment and of the application also need to be promoted at the same time.
- Managing the promotion of certain bits of configuration associated with an application
- The underlying task could be replacing custom configuration files (such as httpd.conf) for your custom web application or updating packages that are required by your application (such as yum and apt-get repositories).

BITS Pilani, Pilani Campus

Orchestration

- Environments are often shared between several applications. This can cause complications in two ways.
 - First, it means you have to take extra care when preparing the environment for a new deployment of an application so as to not disturb the operation of any other applications in this environment.
 - This normally means ensuring that changes to the configuration of the operating system or any middleware don't cause the other applications to misbehave.
- Ensuring that there are no conflicts between the chosen versions of the applications.
- If this turns out to be a complex exercise, you might consider using some form of virtualization technology to isolate the applications from each other.

BITS Pilani, Pilani Campus

Orchestration

- Second, the applications sharing the environment may depend on each other.
- This is common when using a service-oriented architecture. In this situation, the integration testing environment is the first time that the applications will be talking to each other rather than to a test double of some kind.
- Thus, much of the work in the SIT environment involves deploying new versions of each of the applications until they all cooperate.
- In this situation the smoke test suite is usually a fully fledged set of acceptance tests that run against the whole application.

BITS Pilani, Pilani Campus

Deployments to Staging Environments

- Before you let your application loose on unsuspecting users, you should perform
- some final tests in a staging environment that is very similar to production.
- Second, the applications sharing the environment may depend on each other.
- This is common when using a service-oriented architecture. In this situation, the integration testing environment is the first time that the applications will be talking to each other rather than to a test double of some kind.
- Thus, much of the work in the SIT environment involves deploying new versions of each of the applications until they all cooperate.
- In this situation the smoke test suite is usually a fully fledged set of acceptance tests that run against the whole application.

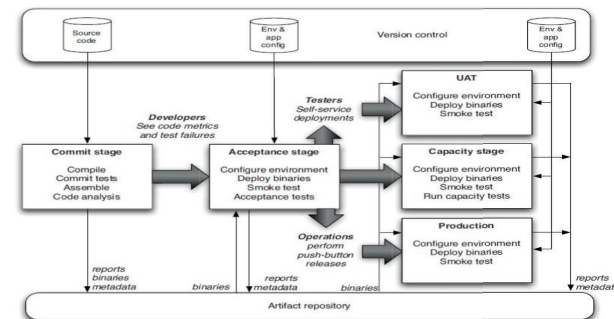
BITS Pilani, Pilani Campus

Deployments to Staging Environments

- You should have started to put your staging environment together at the beginning of your project.
- Ensure your production, capacity testing, and staging environments are commissioned.
- In particular, on a green field project, have your production environment ready some time before the release, and deploy to it as part of your pipeline.
- Have an automated process for configuring your environment, including networks, external services, and infrastructure.
- Measure the warm-up period for your application. This is especially applicable if your application uses caching. Incorporate this into your deployment plan.
- If possible, get your application into its production environment well before release.
- If possible, try rolling your system out to a small group of users before you roll it out to everybody. This technique is known as **canary releasing**
- Deploy every change that passes acceptance tests to your staging environment

BITS Pilani, Pilani Campus

Basic Deployment Pipeline



BITS Pilani, Pilani Campus